BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INTELLIGENT SYSTEMS
ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

DETECTING AI-GENERATED FAKE REVIEWS IN ONLINE PLATFORMS
DETEKCE FALEŠNÝCH RECENZÍ GENEROVANÝCH AI NA ONLINE PLATFORMÁCH

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **IGOR LACKO**
AUTOR PRÁCE

SUPERVISOR **Ing. ANTON FIRC, Ph.D.**
VEDOUCÍ PRÁCE

BRNO 2026

Abstract
This thesis aims to implement a classifier for detecting fake reviews generated by artificial intelligence on shopping platforms, focusing on Slovak reviews. The thesis analyzes the impact of fake reviews on the reputation and sales of products and services. A synthetic dataset is created by scraping Slovak reviews from Booking.com, generating another set of reviews via a language model, and combining them. Then, the thesis presents a method for classifying these reviews. [[**It does this by this and that**]]. [[**The results are this and that**]].

Abstrakt
Táto práca sa zameriava na implementáciu klasifikátora na detekciu falošných recenzií na shopping platformách, so zameraním na slovenské recenzie. Práca analyzuje reputačný a finančný dopad falošných recenzií na produkty a služby. Práca taktiež vytvára syntetický dataset zbieraním slovenských recenzií z Booking.com a ich kombináciou s recenziami vygenerovanými jazykovým modelom. [[**Robí to tak a tak.**]] [[**Výsledky sú to a to**]].

Keywords
Fake reviews, Machine learning, Artificial Intelligence, Text classification, Natural language processing, Python, Large language models

Klíčová slova
Falošné recenzie, Strojové učenie, Umelá inteligencia, Klasifikácia textu, Spracovanie prirodzeného jazyka, Python, Veľké jazykové modely

Reference
LACKO, Igor. *Detecting AI-Generated Fake Reviews in Online Platforms*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Anton Firc, Ph.D.

Detecting AI-Generated Fake Reviews in Online Platforms

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Ing. Anton Firc, Ph.D. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used large language models (ChatGPT, Gemini) for the language revision of the thesis. I have used Github Copilot to assist with the implementation part of the thesis.

....
Igor Lacko
February 24, 2026

Acknowledgements

Contents

List of Figures

Chapter 1

Introduction

Fake reviews have been a problem for online platforms since their inception. Reviews are often the first thing someone reads when considering buying a product or using a service. As such, they are one of the biggest influences in it's reputation, which in turn influences it's sales. Moreover, people may not even find out about a product if it's reputation is damaged by fake reviews, or in a opposite example, if a product is not artificially propped up. It's no wonder that people or companies often try to take advantage of this, whether it's for their advantage via faking positive reviews on their products or to the disadvantage of potential competitors via faking negative reviews on their products. The „traditional" way of faking reviews in the past has been to manually write them, or pay for someone else to do it, which is time consuming and costly due to requiring manual human labor. However, with the rise of generative transformer language models, such as *GPT*, and the ease of access to them, fake reviews can now be mass produced for cheap. The detection of fake reviews, explored in detail in is a continuously developing field. This is due to multiple factors: human written fake reviews differ from AI generated ones, and even among the latter, as language models keep evolving, the methods for detecting their output have to as well. As humans have been shown not to be consistently able to correctly classify reviews as real or fake, as well as manual classification not being feasible from a time perspective, there are multiple benefits to automating this process. A common way to do this is by using machine learning (ML) methods, whether via more „traditional" ML models such as *Naive Bayes*, *Logistic regression* or *Support vector machines (SVM)*, or more recently via utilizing language models. Commonly used features for these models are stylometric ones such as text length or writing style. However, these approaches, introduce another challenge, which is collecting relevant testing and training data. Existing datasets are few and far between, overly represent the hospitality domain and are mostly in English. To my best knowledge, no annotated dataset of Slovak reviews exists as of now. This work aims to create a synthetic dataset in the Slovak language by combining existing reviews written in Slovak, using a generative model to generate the „fake half" of the dataset, and train a classifier on the created dataset for the purpose of detecting reviews generated by language models. The classifier is an ensemble model which utilizes a pre-trained BERT-like model for the Slovak language in addition to classifiers who take in various stylometric features and other textual features, such as TF-IDF. The text of this thesis is structured as follows: Chapter 2 describes the necessary theoretical parts of Machine Learning and Artificial Intelligence needed as the base for the other chapters. Chapter 3 explores the problem fake reviews and what impact they have. Chapter 4 then summarizes the features and techniques used in the domain of fake review detection up to this point, including both traditional machine learning methods and detection using transformers based language models. Chapter 5 describes a high-level overview of the architecture of the proposed classifier. Chapter 6 describes the used dataset, how it was collected, it's characteristics, and how the synthetic half was generated. Chapter 7 describes the classifier in detail, including which models were chosen for each part and the full list of features used. Chapter 8 then discusses the results and describes how they were evaluated, and lastly Chapter 9 concludes the thesis.

Chapter 2

Transformer based language models

Most modern language models are based on the *Transformers* architecture, which was invented by the 2017 paper *Attention is all you need* [35].

2.1 The transformers architecture in general

Language models split the input text into **tokens**, which are one part of a sentence processed by the language model. They are usually words, but they can be parts of words, or even characters. Language models have a vocabulary of tokens they „understand“, where each token has a unique numeric identifier. These identifiers are what the models actually work with. Modern language models can have very large numbers of tokens in their vocabulary, for example OpenAI's *GPT-4* (2023) model had around 100 000 tokens in it's vocabulary, while *GPT-4o* (2024) almost doubled that number[38]. Another important term when it comes to LLMs is the **context window**. This means the maximum amount of tokens that the model can process at the same time. Figure 1, taken from the aforementioned *Attention is all you need* shows the architecture of a transformer model. In it, we can see that the model is split into 2 main parts. These are called the *Encoder* and the *Decoder*, respectively. In essence, the encoder block captures the meaning of the input through embeddings and attention, while the decoder processes them and later converts them into probabilities for each token. Both the encoder and decoder block are composed of a stack of encoders/decoders respectively. Despite the original transformers architecture having both these parts, most models usually only have one. *Encoder-only models* are used for a range of NLP (Natural Language Processing) tasks such as text classification, sentiment analysis, or masked language modeling[1]. Perhaps the most known encoder only model is *BERT* (Bidirectional Representation From Transformers), introduced in [8] and used as a foundation for further encoder only models. *Decoder-only models* are used mainly for text generation and are the underlying engine under popular chat bots such as Chat-GPT, which uses the *GPT* model (Generative Pre-Trained Transformer), introduced in [31]. Models that have both parts are called *Encoder-Decoder models* and are used for text-to-text tasks such as translation or text summarization.

[1] masked language modeling – process where the model is given a input sentence with missing („masked“) parts and tries to fill in those parts

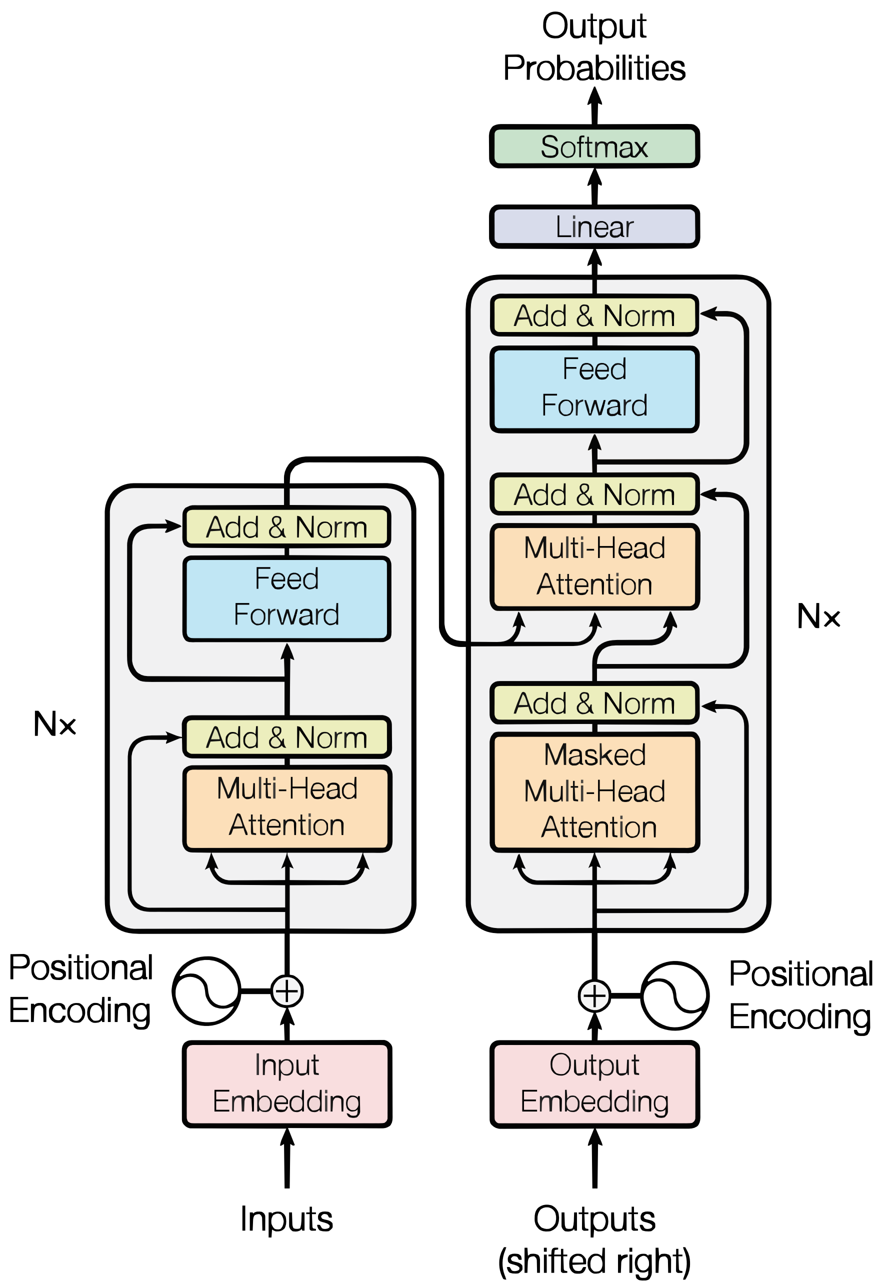

Figure 1: The transformers architecture.

2.2 Word embeddings and attention

What makes transformers-based models so effective is the attention mechanism. Like language models before them, transformers use *word embeddings*, which in essence are high dimensional vectors which numerically represent each input token (1 token = 1 embedding), where their values represent the semantic meaning of the text in the high-dimensional space. A well-known example are the words „king“ and „queen“. If you subtract the embedding for „man“ from „king“ and add „woman“, you get a vector that is very close to „queen“ in the high-dimensional space. One of the best known approaches used to create word embeddings was *word2vec* [21], introduced at Google in 2013, which uses a neural network. These embeddings are learned, and therefore, fixed representations of the input. Another classic example which represents what this means is the word „bank“. Since the embeddings are static, the embedding for this word will be the same for the sentences *„They robbed a bank“* and *„They were sitting on the river bank“*. What attention does is modify the embeddings as they pass through the transformer, based on the words around them and giving more weight to tokens that may be more relevant to the currently processed one, allowing the model to *attend* to them with more importance. In other words, it gives *context* to the embeddings, so a transformers model will use different embeddings for the word „bank“ in the aforementioned examples. These embeddings are called *contextualized word embeddings*[14], and are primarily used by transformer based language models.

2.3 Decoder models

Decoder-only models are the standard for generative NLP tasks. They work in a autoregressive manner, by taking in the input text (the prompt) and predicting the next most likely token to appear after it. The given token is then generated, and appended to the prompt. The prompt with the generated token is then used as the input for the next cycle. The model does this until it generates the entire output.

Predicting the next token

In each cycle, as a prompt passes through a decoder, the model outputs a long list of scores (the size of it's vocabulary, which is the list of tokens „it knows“) called *logits*. These scores are normalized so that they are all positive and add up to 1 using a function called *softmax*[2]. The model then chooses the token with the highest probability as it's output. The softmax equation, taken from [37] is defined as:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

where z is the vector of logits and z_i is the *ith* logit.

System prompts

Of course, only this approach is not enough for example to get chat bots to generate concise answers which look like answering a question like most modern models do. This is why language models have additional *system prompts*, which serve as a static set of instructions or general guidelines to the model and is taken as an input alongside the user prompt. For example, a chat bot like Chat-GPT may have a system prompt which looks like: *„You are* *a helpful chat bot which talks to people and answers their questions. The following text is the user's input.“*. Then, the user prompt would be concatenated to this to produce the whole prompt.

Temperature

Since decoder models predict the next most likely token, they should theoretically, for the same input always give the same output (this isn't practically always true since other hyper parameters like *top_p* or *top_k* which influence how the top results are sampled can also affect randomness [6]). However, sometimes we want the model to show non determinism, whether it is to appear „more human“, or to give a wide range of various results when generating synthetic data. *Temperature* is a parameter mostly between 0 and 2 (although some platforms/models have a different range similar to this) which controls how random the model's output is. This works by dividing each logit by the temperature when computing the softmax equation[37], which results in higher differences between the probabilities with lower temperatures.

2.4 Encoder models

Due to the encoder component being designed to capture the understanding of the input text, encoder models are used for tasks which require this. The input of the encoder are vectors of embeddings created from the input sentence (for each token). The output is then another vector with context of the input sentence „baked“ into it using the attention mechanism. In the standard transformers architecture, this output is then passed into the decoder block. However, there is no decoder in modern BERT like models, so the usage of the output can vary. For example, the BERT paper [8] describes several ways to use the output vectors. The model was originally trained on masked language modeling. 15% of tokens were randomly chosen for masking. The tokens were replaced with a special *[MASK]* token with 80% probability, a random token with 10% probability and stayed unchanged with 10% probability. The output of the final encoder which corresponds to the position of the masked token is ran through a softmax, and the word with the highest probability is chosen as the „original“ word which was masked before. For classification, the model always has another special *[CLS]* token as the first token of the sentence. This token represents the „aggregate“ representation of the input, and is used as the input for the final layer of the model. This final layer can be task-specific, and is often called the *classification head*, which can be task-specific (as the base model does not have this), but is usually another neural network [23].

Chapter 3

Fake reviews

Fake reviews pose a huge problem to both customers and sellers. They reduce the trust of customers in online platforms and thus have an inherently negative commercial impact on online available products. The following section discusses the importance of reviews and the possible impacts of fake reviews.

3.1 The influence of fake reviews

Fake reviews can have various negative effects. For example, a customer may not buy a product because of negative reviews associated with it. This is obviously unjust if there is a disinformation campaign against the product. On the other side, a product may get higher sales due to being artificially propped up by positive reviews, as a tactic of the seller to gain a market advantage over the competition.

Influence on customer purchasing behavior

Perhaps the biggest factor in deciding whether to buy something is it's reputation. That is why reviews exist, to present that information to people to better decide. The value of this is rooted in the presumed truthfulness of the reviews [10], which spammers[1] take advantage of. This doesn't only impact products ordered from the internet. [36] shows that around 93% of consumers say that online reviews influence their choice. More recently, [27] found that more than 70% of consumers read business reviews regularly when searching for local businesses and another 26% do so occasionally. Fake reviews can impact customer purchasing behavior in several ways. Firstly, a product with positive fake reviews can lead customers to buy it more, which can be harmful as fake reviews don't provide anything truthful regarding the value of the product. On the other side are negative reviews, which can cause consumers to distrust the product and also the seller [16]. Lastly, the knowledge of fake reviews existing itself can lead customers to distrust the review system itself and doubt all reviews [39], which can lower their purchasing intentions, indirectly harming the sales of online products or services.

Commercial impact of reviews in general and fake reviews

As the prior section implies, reviews can influence consumers purchasing behavior. This, in turn, can have real financial impacts on the reviewed products. Several studies show

[1] spammer — a person who posts fake reviews [24]

correlation between review ratings and sales. For example, [18] shows that a 1 star increase in Yelp rating leads to around a 9% increase in revenue, as an example of what may lead spammers to generate untruthful reviews. Considering only fake reviews, a 2021 study by the World Economic Forum [20] found that the impact of fake reviews on online spending is around 152 billion. They based this on the total influence of reviews, which, according to a study they cite is was 3.8 trillion in 2021 and their finding on average percentage of fake reviews on e-commerce sites, which they found to be 4%. This study also says that the eventual culling of fake reviews happens in general around after 100 days. This shows that fake reviews may even be a low-risk/high-reward situation for businesses who try to promote themselves throughout them, since the considerable gains outweigh the comparatively low risk. More recently, [32] gathered statistics concerning fake reviews for 2025. They estimate that fake reviews cost customers more than 700 billion in 2025, that around 30% of online reviews are considered fake, and that more than 80% of customers encounter them. Interestingly, they found that the number of fake reviews grows about 12% faster than the number of other reviews, which may indicate increased AI usage for fabricating fake reviews. Another interesting thing they found is that 46% of the reviews considered fake have been positive in sentiment (5 stars). This correlates with other studies, such as [10] which also attribute overly positive sentiment as a possible indication of fake reviews.

3.2 Generating fake reviews using AI

In the past, fabricating fake reviews was usually done by manually writing them or buying them. Several earlier studies, such as [13] show that there exists a market for fake reviews. As the cited study says, a popular communication channel for procuring fake reviews were private Facebook groups. Nowadays, malicious actors have another method of obtaining fake reviews – using generative language models. There are many options here, such as using available APIs (for example OpenAI's) for text generation, or just using a pre–trained generative model from the Hugging Face hub for review generation, or even just opening a language model web interface and asking to generate a review. [15] analyzed the frequency of AI-generated google reviews. They found that the rate nearly doubled from 2023 to 2024, and by 2024 almost one in five reviews on google is AI-generated. In addition to that, the quality of AI-generated reviews may be often higher than manually fabricated ones. Several studies have shown that humans have trouble distinguishing AI-generated reviews from real ones, often being no better than a random guesser (around 50%). Furthermore, studies discussing manually fabricated fake reviews talk about the inability of spammers to express true sentiment, which distinguishes their reviews from true ones. Large generative models whose training data includes reviews may not have this limitation. Another thing to consider is the ease of generating fake reviews. Manually writing fake reviews is time consuming, and the result is small in quantity. Just an online LLM can easily output 10 reviews in a single prompt. A python script controlling a loaded model for inference can output thousands in a tiny fraction of the time it would take to write them manually depending on the spammer's hardware.

Chapter 4

Detecting AI generated reviews

Detection of AI-generated reviews is a field that is still very much in development. Despite the first research papers concerning fake review detection in general (e.g. including human-written fakes) dating back years, the number of studies concerning fake reviews has largely increased, and specifically concerning AI-generated reviews. Virtually all today's studies utilize some form of Machine Learning, whether it be traditional ML models or Deep Learning models.

4.1 Commonly used features in fake review detection

Stylometric features

Stylometry is the study of writing style. Several studies utilize it for feature extraction, since language models seem to have different writing style than humans. Some of the most common stylometric features are the frequency of using punctuation, average sentence or word length, n-gram features, or how often the review repeats words ([24], [11]). In simple terms, stylometry analyzes the habits of the writer during creating text (which is why it is also often used in authorship verification [29]).

Concerning unique words, one of the best known features concerning lexical diversity is the *Type-Token Ratio (TTR)* [19]. It is defined as:

$$TTR(text) = \frac{|unique\_tokens(text)|}{|tokens(text)|}$$

It's a number from $< 0, 1 >$ indicating how lexically diverse the input text is. It could be useful to detect a model using a limited set of words.
Another useful metric which measures the simplicity of a input text is the *Automated Readability Index (ARI)*. Parts of the following paragraph about it are paraphrased from [10]. For a given text, ARI is defined as:

$$ARI(text) = 4.71\frac{|chars(text)|}{|words(text)|} + 0.5\frac{|words(text)|}{|sentences(text)|} - 21.43$$

ARI is a readability metric which indicates the US education level needed to understand the text. A score around 3 means that a US third grader can understand it, etc. It is similar to TTR in that a higher number usually indicates a more complex text.

N-grams are sequences of elements of length N. „Elements“ most usually refers to words here, but there can be character or token based N-grams [1]. Unigrams have 1 element, bigrams have 2, and so on. For example, if we generalize elements to words, and ask which bigrams are in the sentence "*Cats are very cute*", they would be:

$$\{(Cats, are), (are, very), (very, cute)\}$$

N-grams can be useful features for finding out which phrases are the most common in the input text. For example, if a language model tends to repeat the same phrases often, the n-grams corresponding to these phrases will appear more often than other ones. This could be especially useful in something as specific as fake review detection, where the models used to generate spam could tend to repeat the same positive or negative phrases.

TF-IDF (Term Frequency-Inverse Document Frequency) is a feature extraction method. It most usually works with individual words, but it can be character level or n-gram level. The individual elements are called *terms*. Parts of the following paragraph are taken from [7]. To compute the vector representation of the input text, TF-IDF uses two vectors. For a term t and a dataset entry E, the TF vector is defined as:

$$TF(t, E) = \frac{\text{count of t in E}}{\text{count of all terms in E}}$$

In simple terms, it's a score which is indicating how relevant t is in E. The more often it appears, the higher the TF score will be.

The second vector, IDF is composed of elements defined for a term t as:

$$IDF(t) = log(\frac{\text{count of entries in dataset}}{\text{count of entries in the dataset where t appears}})$$

This vector assigns a higher score to terms which appear in less documents, hence the *inverse*. As opposed to TF, this vector is mutual for the entire dataset, and as such does not need to be computed for every text inside.

The resulting vectorized text, TF-IDF is then defined as:

$$TF - IDF(t, E) = TF(t, E) * IDF(t)$$

meaning that the highest score terms are the ones, which are relevant to that specific entry but don't appear too often in other entries.

Semantic features

Semantic features focus on the meaning of the text instead of it's structure. This can allow them to model relationships between words that aren't seen just from the structure, or extract deeper meaning from the review.

A common semantic feature is the *sentiment* of the review. Per multiple studies ([34], [12], [10]) polarized sentiment is prevalent in fake reviews. That means that fake reviews tend to be very positive or very negative, but very few „in the middle“. Per [12] however, positive fake reviews outnumber negative fake reviews (which might be expected, since promoting own product seems like it would have more influence than defaming the competition). Review sentiment can be extracted in various ways, such as lexicon based tools [34] or by using a sentiment classifier.

Word embeddings, explained previously in section 2.2 are a widely used method to capture semantic features. The underlying features are more of a „black box" to us, since we don't know what the individual dimensions in the embedding vector space represent, but they are a very effective method to turn text into numbers and capture more than it's structure. Word embeddings are primarily used as features with deep learning models (various neural network architectures, Transformer-based models) but can be used as features with other classifiers as well. For example, [22] used a BERT model to extract contextualized word embeddings, and then trained multiple classifiers on these embeddings, such as SVM, Random Forest, and more.

Review and product metadata

Review metadata are features about the review, but not about it's text. They go hand in hand with metadata about the product. Examples of this include review rating, product ID, or date and time posted ([24], [11]). One of the aforementioned studies ([10]) which talks about the polarized sentiment of fake reviews also found that fake reviews tend to follow a bimodal rating distribution, meaning they have sharp peaks at the lowest and highest rating. Other studies have found that other ratings of the product could also be a useful feature, with [33] finding that the number of 1 star ratings on a product was one of the most important features in their Random Forest model.

Reviewer centric features

Reviewer centric features are characteristics of the reviewer's profile. [24] lists some of these features. They include features such as repeated reviews across products reviewed, sentiment of the reviews (for a concrete reviewer), or burstiness (tendency to post a lot of reviews in a short time span). Another survey [11] also lists the absence of profile data, or review time span.

4.2 Traditional machine learning models in fake review detection

SVM

Random Forest

Naive Bayes and it's variants

4.3 Large language models in fake review detection

Chapter 5

Architecture of the classifier

For the classifier, a stacked ensemble model was chosen with 3 inner models. Each of the models focuses on different features of the dataset, and their outputs are fed as inputs to the meta learner, who makes the final prediction. Figure 2 displays a high level overview of the proposed architecture.

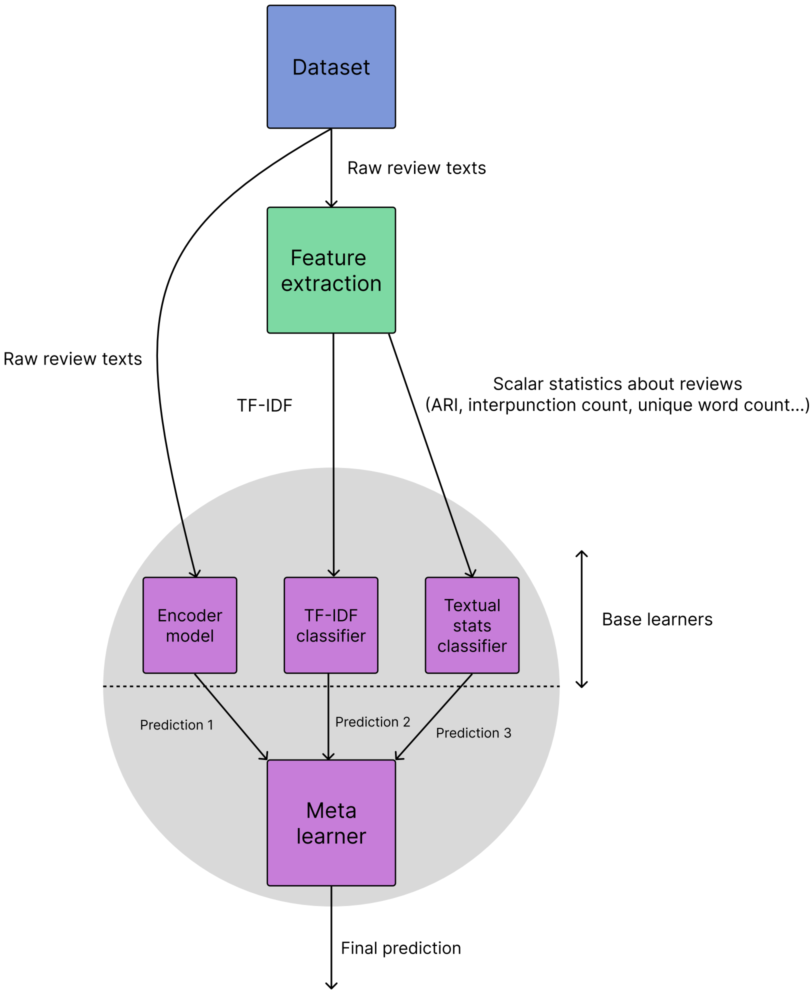

Figure 2: Architecture of the fake review detector.

The base learners are trained on either the raw review dataset, or vectors of features which are extracted from it to be used as input to the model. The meta learner is then trained on their predictions, which are numbers indicating how likely a review is to be real or fake. The meta learner's final prediction has the same meaning.

5.1 Model components

Encoder language model

One of the base learners is a encoder language model fine-tuned for review classification. The purpose of this component is to find semantic giveaways and patterns in the text which could indicate the text being artificially generated. Since this is a language model, it is trained on the raw review texts and the class labels, (which are internally turned into embeddings and then contextualized) and takes the raw text as input during inference.

TF-IDF classifier

Another base learner is a classifier which tries to predict the authenticity of the reviews upon their TF-IDF score. This is split from the textual stats classifier since the TF-IDF vector will be much larger than the vector of all other features, which are scalar values, as not to „drown out“ the other features.

Textual features classifier

The final base learner is a model which tries to classify the review based on textual features which are scalar values. This includes features such as ARI, TTR, count of punctuation characters, average sentence length, and others. The full list of features is provided in appendix A.

Meta learner

This is the part of the model which has „the final say“ about the authenticity of the review. It takes in the predictions of the base models and outputs the probability of the review being AI generated.

Chapter 6

Dataset

As mentioned in the introduction, to my best knowledge there is no annotated Slovak dataset of reviews. For this reason, a new dataset was collected using reviews of Slovak hotels on Booking.com. From the scraped reviews, two variants were created. One where each review was used as „unified" content (meaning the positive and negative content were concatenated) and one where they were used as separate samples.

6.1 Choice of the data source

Part of the reason why Booking.com was chosen is due to inspiration by previous datasets used in fake review detection. The hospitality sector dominates the percentage of the used datasets, per [24] and [11]. Figure 2, taken from [11] and recreated into a svg shows the distribution of the datasets used from 2021 to 2023:

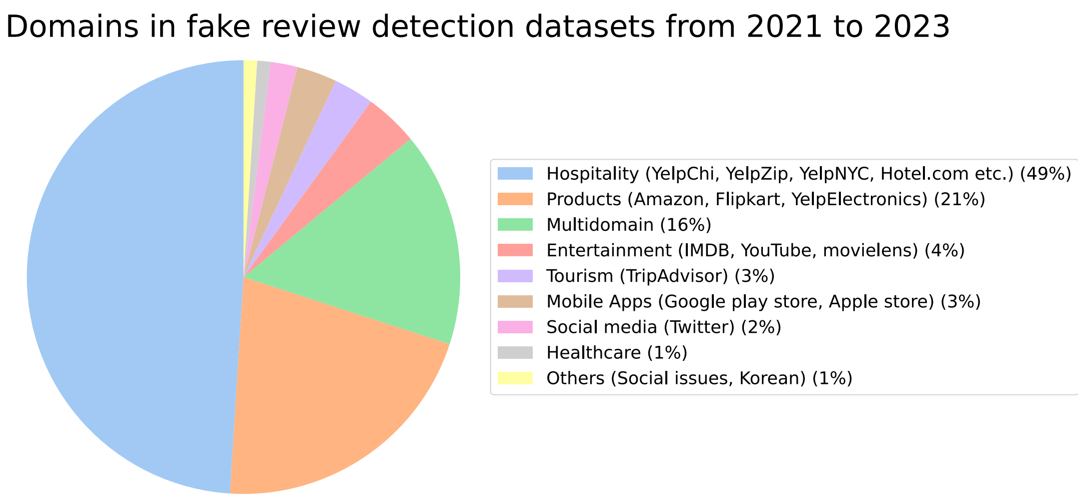

Figure 3: Distribution of dataset domains used in fake review detection from 2021 to 2023

Booking.com represents the largest source of reviews of hotels all over Slovakia, with options to filter by language to use only reviews in Slovak. The website can be easily parsed, and the reviews are structured into 2 parts (the „good part", or „what did you like?", and the

„bad part“, or „what did you dislike?“) which is more complicated than 1 review = 1 text, but allows for multiple variants (use a review as one sample or as 2 samples).

6.2 Collecting data

The process of scraping

A total of 10064 reviews were scraped from the Booking.com website. The scraper always navigated to the website and searched for hotels in Slovakia. Then it would go hotel by hotel, filter the reviews by language to Slovak, and scrape each review from a hotel which it didn't scrape already (known by URL). The scraper would extract the hotel name (the individual CSV files were saved by their hotel name for filtering already scraped hotels), the positive content, negative content and rating and save them into a CSV.

Scraping permission and legal aspects of the used data

Since web scraping is a legally gray area, and Booking.com's terms and conditions [4] disallow any sort of monitoring, scraping and crawling, permission was asked from their data privacy office. They granted it under the conditions that the collected data is anonymized, the review content is not refered to, and the data is not shared. Per their request, the dataset does not contain any personal data, and it's content is not published other than aggregate statistics.

6.3 Dataset characteristics

This section describes the non synthetic part of the dataset, it's characteristics and statistics. The dataset contains 10064 rows, each of which is can be defined as a quadruple of $(rating, content_{good}, content_{bad}, content_{unified})$ where the last is just the two content parts concatenated with a whitespace between them if both are present (else it's identical to the present part). Table 1 describes how many of the scraped reviews contain only positive and negative content, and how many contain both.

Content type	Count
Total reviews	10064
Only positive content	2587
Only negative content	370
Both positive and negative content	7107

Table 1: Statistic of content type presence in the reviews.

From the table we can see that about three quarters of the dataset have both negative and positive content. Out of those that don't, there are almost 7 times as many reviews with only positive content than reviews with only negative content. From this, it comes as no surprise that the dataset heavily skews towards positive ratings. Booking.com uses ratings in the interval $< 1, 10 >$ and figure 3 shows that the mean rating is 8.24, and if we go by discrete bins, reviews with a rating of 10 are by far the most common. This might be due to Booking.com pushing highly rated hotels up as default results in a location.

Figure 4: Distribution of ratings in the dataset.

When it comes to the text length, figure 4 and table 2 show that both the positive and negative parts of the reviews are mostly around 110 characters (in median), with long outliers skewing the mean to the right, but both parts don't significantly differ from each other.

Content type	Average length	Median length	Max length	Min length
Positive	148.65	117.00	1988.0	1.0
Negative	159.18	106.00	1998.0	1.0
Total	262.15	195.00	2810	96

Table 2: Statistics of review lengths for different content types.

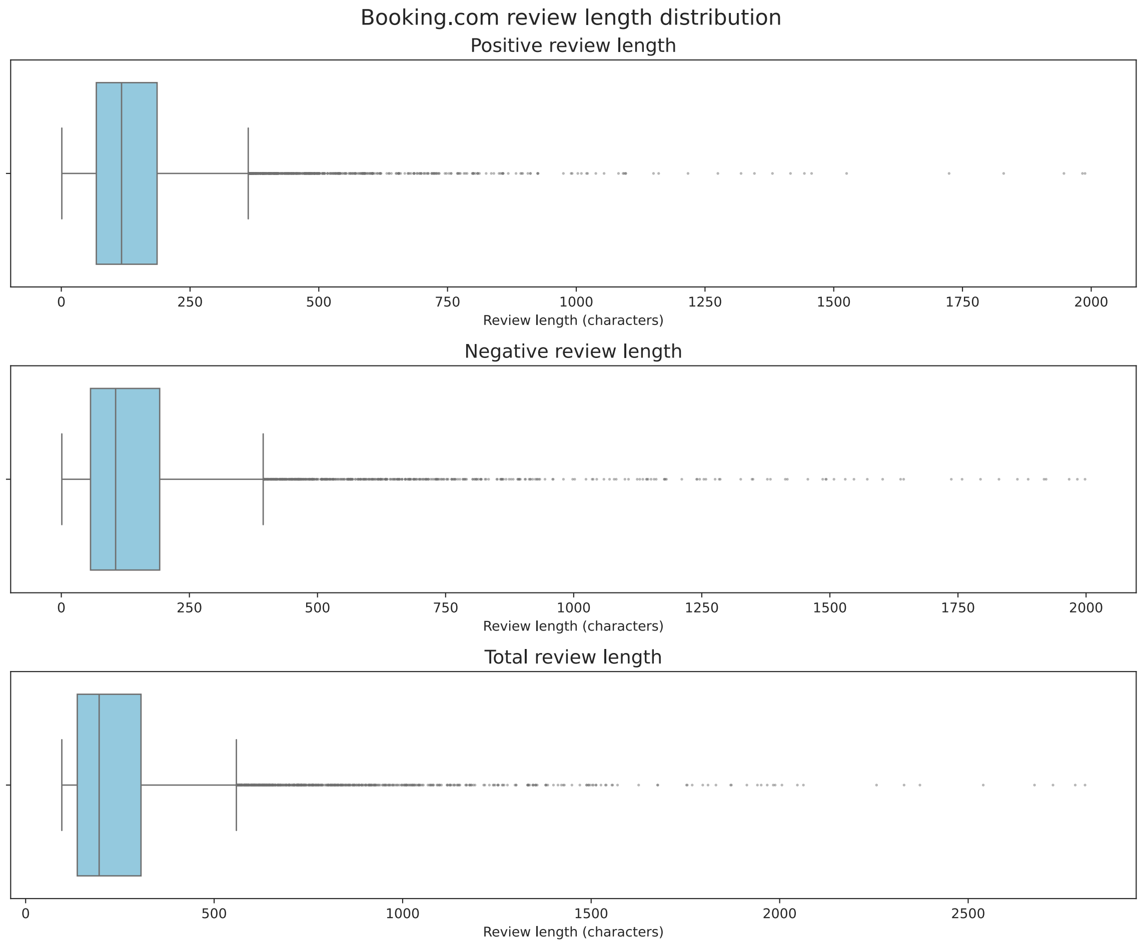

Figure 5: Distribution of review lengths in the dataset.

6.4 Generating synthetic data

Datasets used in the past have been constructed by methods such as crowdsourcing [24] or some other form of asking people to manually fabricate fake reviews. Another commonly used method was by using a filtering algorithm. One of the oldest datasets for fake review detection [25] collected hotel reviews from Yelp, which has it's own filtering algorithm but doesn't delete reviews labeled as spam, but puts them in a separate list. However, the first method makes it difficult to gather large amounts of data, while the second labels reviews based on a non transparent algorithm. Another option is to create a new, synthetic dataset based on the „real" one which contains scraped reviews. This involves using a generative language model to generate reviews and annotating them as fake. [10] which used GPT-3 and [17] which used various models have found success with this method. It's benefit is that it allows the generation of high quality annotated data very quickly, however we still need the „real" part of the dataset.

Model comparison

For generating synthetic reviews, 3 models were evaluated. Two of them were instruct models. The first was a 27b version of the Gemma 3 model by Google, and the second was a

32b version of the Qwen 3 model by Qwen [30]. Both of those are known for their multilingual capabilities. The third model was the mistral-sk-7b model [26], which is a fine-tuned mistral model for the Slovak language. This model is a base model. Each of these models was used to generate 200 samples from the unified variant of the dataset, and 200 samples from the separate. Then the generated text quality was compared with 2 methods:

1. Perplexity: For each of the generated sample „datasets", perplexity was computed on the whole set using an independent model.
2. Survey: A Google Forms survey was created, asking humans to rank the models by how „natural" the text is.

Model prompts

The generative models were always prompted with example reviews from the dataset. To make the synthetic data follow the sentiment distribution of the real dataset, the examples were sampled as follows. One review was always randomly sampled from the entire dataset. Then:

1. For the separate variant: 5 examples were randomly chosen from the part of the dataset that matched it's content type (positive /negative). The content type was stored alongside the new rating for later stratification into training/validation/testing sets.
2. For the unified variant: 5 examples were randomly chosen from the part of the dataset whose rating was in $< max(rating_{example} - 1, 1), min(rating_{example} + 1, 10) >$. The rating of the example was also stored alongside the new review.

The prompts itself however differed between models. Since the mistral model is not a instruct model, which means it's not tuned to follow instructions, we couldn't use a prompt like *„Vygeneruj recenziu hotela po slovensky..."*. Instead, the model was always left to fill in the last example. The generic prompt for this model then looked like:

```
Hotelová recenzia 1: <Example 1>
Hotelová recenzia 2: <Example 2>
...
Hotelová recenzia 6:
```

On the other hand, the instruct models allowed bigger flexibility with the prompt. Based on the sampled content type of rating, the prompt was always a bit different. The models followed a label instruction, which described what should the review's sentiment look like. This is showcased in tables 3 and 4 for the unified and separate variant.

Rating	Label instruction
≥ 8	veľmi spokojná
≥ 6	spokojná
≥ 3	nespokojná
< 3	veľmi nespokojná

Table 3: The label instructions based on the sampled rating.

Content type	Label instruction
$content_{good}$	o tom čo sa hosťom páčilo
$content_{bad}$	o tom čo sa hosťom nepáčilo

Table 4: The label instructions based on the sampled content type.

The models also produced much longer reviews than the given examples. To mitigate this, the min review word count, max review word count and average review word count were computed from the examples and included in the prompt, as:

```
f"Dĺžka má byť približne {target_words} slov "
f"(rozsah {min_words}-{max_words} slov). "
f"Nepíš viac ako {max_words} slov.
```

Model evaluation

Perplexity

Perplexity (PPL) is a generative LLM evaluation metric, which describes how much a model is confused („perplexed“) by a text, in the sense that it outputs how many tokens was it choosing from on average [5], which means that lower perplexity usually indicates more coherent text. It is computed as ([9]):

$$PPL(X) = exp(-\frac{1}{t}\sum_{i=1}^{t} \log p_\theta(x_i|x_{<i}))$$

Where X is the text to compute perplexity for (in this case, one review), t is the length of the review in tokens, and $p_\theta(x_i|x_{<i})$ denotes the probability that a the *ith* token would follow the preceding tokens.

The perplexity was computed for each text in the sample datasets, summed up and then averaged. To avoid a situation where a model would assign itself a lower perplexity score, a independent model was used to compute this. The model used was the EuroLLM-22B-Instruct-2512 model [28], which has Slovak in the list of languages it understands. The results are visible in table 5.

	Model	Perplexity	Variant
0	Gemma	7.107292	Unified
1	Qwen	17.466667	Unified
2	Mistral	17.810422	Unified
3	Gemma	14.902512	Separate
4	Qwen	73.536782	Separate
5	Mistral	47.902919	Separate

Table 5: Perplexity scores for the individual models for each variant.

For both variants, the Gemma model has the best score. The other two models are close in the unified variant, but the mistral model is a distant second in the separate one.

Google Forms survey

The other technique used to evaluate the generated text was to let human evaluators read it and rank the models. A Google Forms survey was created with 10 questions. Each question contained one sample from each model, where 5 of the questions contained samples from the unified variant and 5 from the separate. The evaluators were asked to rank the examples from most to least natural sounding, e.g. to rank the one where they would most likely believe that it was human written at the top, and the opposite at the bottom. Each model has assigned a point for a first place vote, half of a point for a second place vote, and zero points for a third place vote. So the total score the model received was:

$$Score = votes_{first} + \frac{votes_{second}}{2}$$

The results are visible in table 6.

	First place votes	Second place votes	Third place votes	Total score
Gemma	39	25	46	51.5
Mistral	49	36	25	67
Qwen	22	49	39	46.5

Table 6: Results of the Google Forms survey.

As opposed to perplexity, the Mistral model wins this by a sizable margin, with the second (Gemma) and third (Qwen) model being pretty close to each other.

Model choice

In the end, the Gemma model was chosen instead of the Mistral model. The Qwen model was disqualified by placing last in both metrics. The Gemma model was chosen due to several reasons:

1. Larger model: The Gemma model is more than 4 times larger than the Mistral model. This reflects in the generated texts where this model is able to be semantically richer and generate longer reviews which are still in coherent Slovak.

2. Instruct model: This makes tweaking the synthetic data to match the real data much easier, since things like expected length or sentiment can be inserted into the prompt.

6.5 Splitting the data for training and testing

Good splits of the data into training, validation and test splits are important to avoid problems during model training such as overfitting. They depend on the size of the dataset, and how balanced it is among other factors. A common ratio is splitting it into 80% training, and 10% per the validation and test splits [3]. Per the cited article, there are multiple techniques to choose „which data goes into each set“. If the dataset is balanced, meaning there isn't one group of the data larger than the other random sampling should suffice, however if the sets aren't equal, stratified sampling might be better. That means that the percentage of the data in each set for a given group corresponds to the total percentage of the group in the dataset. The dataset is balanced when it comes to the volume of real and synthetic data, however it's not balanced when it comes to sentiment. For this reason, the data is sampled using the stratified strategy. As mentioned in section 6.4, the chosen rating/content type was stored alongside the generated review. Then, when creating the training, validation and test splits, they were used as variable for their variant respectively alongside the class label (whether the review is real or synthetic) for stratified sampling. Another thing to consider, is that with ensemble models it needs to be decided which data to use for the training of the meta model. We could train the model on the predictions of the base models on the same data they were trained on, but those could be inflated since they were trained directly on them. A common method

Chapter 7

Implementation of the classifier

Chapter 8

Evaluation

8.1 Evaluation process

8.2 Results

8.3 Discussion and practical applications

Chapter 9

Conclusion

Bibliography

[1] Ahmed, H.; Traore, I. and Saad, S. Detection of Online Fake News Using N-Gram Analysis and Machine Learning Techniques. In:. October 2017, p. 127–138. ISBN 978-3-319-69154-1.

[2] Alamar, J. *The Illustrated Transformer*. 2018. Available at: https://jalammar.github.io/illustrated-transformer/.

[3] Baheti, P. *Train Test Validation Split: How To Best Practices*. 2024. Available at: https://www.v7labs.com/blog/train-validation-test-set.

[4] Booking.com. *Terms and Conditions*. 2025. Available at: https://www.booking.com/content/terms.html.

[5] Brenndoerfer, M. *Perplexity: The Standard Metric for Evaluating Language Models*. 2025. Available at: https://mbrenndoerfer.com/writing/perplexity-language-model-evaluation-metric.

[6] Brenndoerfer, M. *Why Temperature=0 Doesn't Guarantee Determinism in LLMs*. 2025. Available at: https://mbrenndoerfer.com/writing/why-llms-are-not-deterministic. Accessed: 2026-01-15.

[7] Das, M.; K., S. and Alphonse, P. J. A. *A Comparative Study on TF-IDF feature Weighting Method and its Analysis using Unstructured Dataset*. 2023. Available at: https://arxiv.org/abs/2308.04037.

[8] Devlin, J.; Chang, M.-W.; Lee, K. and Toutanova, K. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. Available at: https://arxiv.org/abs/1810.04805.

[9] Face, H. *Perplexity of fixed-length models*. Available at: https://huggingface.co/docs/transformers/perplexity.

[10] Gambetti, A. and Han, Q. *Combat AI With AI: Counteract Machine-Generated Fake Restaurant Reviews on Social Media*. 2023.

[11] Gupta, R.; Jindal, V. and Kashyap, I. Recent state-of-the-art of fake review detection: a comprehensive review. *The Knowledge Engineering Review*, 2024, vol. 39, p. e8.

[12] Hajek, P.; Hikkerova, L. and Sahut, J.-M. Fake review detection in e-Commerce platforms using aspect-based sentiment analysis. *Journal of Business Research*, 2023,

vol. 167, p. 114143. ISSN 0148-2963. Available at:
https://www.sciencedirect.com/science/article/pii/S0148296323005027.

[13] He, S.; Hollenbeck, B. and Proserpio, D. The Market for Fake Reviews. *Marketing Science*, 2022, vol. 41, no. 5, p. 896–921.

[14] Jay Alamar, M. G. *Hands-On Large Language Models*. O'Reilly Media, 2024.

[15] Lambert, M. *From 2019 to 2024: AI-Generated Google Reviews Increased by 279.2 2025. Available at: https://originality.ai/blog/ai-google-reviews-study.*

[16] Le, *L. H. and* Ha, *Q.-A. Effects of negative reviews and managerial responses on consumer attitude and subsequent purchase behavior: An experimental design.* Computers in Human Behavior*, 2021, vol. 124, p. 106912. ISSN 0747-5632. Available at: https://www.sciencedirect.com/science/article/pii/S0747563221002351.*

[17] Liu*, M. and* Poesio*, M.* Data Augmentation for Fake Reviews Detection in Multiple Languages and Multiple Domains*. 2025. Available at: https://arxiv.org/abs/2504.06917.*

[18] Luca*, M. Reviews, Reputation, and Revenue: The Case of Yelp.Com.* SSRN Electronic Journal*, september 2011.*

[19] Majumdar*, P. Type-Token Ratio (TTR) - A Measure of Lexical Diversity, june 2025.*

[20] Marciano*, J. Fake online reviews cost* $152 billion a year. Here's how e-commerce sites can stop them. World Economic Forum$, 2021.

[21] Mikolov, T.; Chen, K.; Corrado, G. and Dean, J. *Efficient Estimation of Word Representations in Vector Space*. 2013. Available at:
https://arxiv.org/abs/1301.3781.

[22] Mir, A. Q.; Khan, F. Y. and Chishti, M. A. *Online Fake Review Detection Using Supervised Machine Learning And BERT Model*. 2023.

[23] Moazeni, S. *Neural Network Heads*. 2025. Available at:
https://medium.com/@AILearningHub/neural-network-heads-16ce560c0b10.

[24] Mohawesh, R.; Xu, S.; Tran, S. N.; Ollington, R.; Springer, M. et al. Fake Reviews Detection: A Survey. *IEEE Access*, 2021, vol. 9, p. 65771–65802.

[25] Mukherjee, A.; Venkataraman, V.; Liu, B. and Glance, N. S. Fake Review Detection : Classification and Analysis of Real and Pseudo Reviews. In:. 2013. Available at: https://api.semanticscholar.org/CorpusID:15124465.

[26] nlp slovak. *Mistral-sk-7b*. 2024. Available at:
https://huggingface.co/slovak-nlp/mistral-sk-7b.

[27] Paget, S. *Local Consumer Review Survey 2025*. 2025.

[28] Project, U. *EuroLLM-22B*. Available at:
https://huggingface.co/utter-project/EuroLLM-22B-Instruct-2512.

[29] PRZYSTALSKI, K.; ARGASIŃSKI, J. K.; GRABSKA GRADZIŃSKA, I. and OCHAB, J. K. Stylometry recognizes human and LLM-generated texts in short samples. *Expert Systems with Applications*. Elsevier BV, january 2026, vol. 296, p. 129001. ISSN 0957-4174. Available at: http://dx.doi.org/10.1016/j.eswa.2025.129001.

[30] QWEN. *Qwen3-32B*. Available at: https://huggingface.co/Qwen/Qwen3-32B.

[31] RADFORD, A. and NARASIMHAN, K. Improving Language Understanding by Generative Pre-Training. In:. 2018. Available at: https://api.semanticscholar.org/CorpusID:49313245.

[32] SHOPPING, C. O. *Fake Review Statistics*. 2025. Available at: https://capitaloneshopping.com/research/fake-review-statistics/.

[33] THEUERKAUF, R. and PETERS, R. Detecting Fake Reviews: Just a Matter of Data. In:. January 2023.

[34] USMAN, U. Identifying Fake Reviews on Amazon Through Sentiment and Linguistic Cues, august 2025.

[35] VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L. et al. Attention Is All You Need. *CoRR*, 2017, abs/1706.03762. Available at: http://arxiv.org/abs/1706.03762.

[36] VIMALADEVI, K. and DHANABHAKAYM, M. A study on the effects of online consumer reviews on purchasing decision. *Prestige International Journal of Management & IT-Sanchayan*. Prestige Institute of Management, 2012, vol. 1, no. 1, p. 91.

[37] WANG, K. *A Comprehensive Guide to LLM Temperature*. 2025. Available at: https://medium.com/@kelseyywang/a-comprehensive-guide-to-llm-temperature-%EF%B8%8F-363a40bbc91f.

[38] YANG, J.; WANG, Z.; LIN, Y. and ZHAO, Z. *Problematic Tokens: Tokenizer Bias in Large Language Models*. 2024. Available at: https://arxiv.org/abs/2406.11214.

[39] ÖZAYDIN, H. Fake Reviews and Ratings Undermining Consumer Trust. In:. March 2025. ISBN 978-625-5958-72-3.